Residual Graph Indexing

What GraphRAG loses, how we get it back, and what the simulation shows

Project review • GraphRAG X-Ray • September 16, 2026

1. The story in four steps

Step 1 — GraphRAG never reads your documents at answer time
It reads *summaries*. While building the index, it groups entities into communities and writes one summary per community. A broad question is answered by reading those summaries, not the original graph.

Step 2 — Writing a summary throws facts away
Two things get dropped, and nobody is told:

- **Boundary loss.** A relationship between two entities in *different* communities belongs to neither summary. It is simply not written down.
- **Budget loss.** Each summary gets 8,000 tokens of input. Bigger communities don't fit, so GraphRAG sorts relationships by how well-connected they are and cuts the list when the budget runs out. The least-connected facts go first — and those are usually the specific details a question needs.

Step 3 — Climbing the hierarchy only moves the loss
Small communities rarely overflow the budget, but they have many boundaries. Merging them removes boundaries but makes each summary compress far more. In our simulation, the leaf level loses 248 relationships at boundaries and none to the budget; the root level loses none at boundaries and 775 to the budget.

> **Choosing a level decides *where* the loss happens. It does not decide *whether* it happens.**

Step 4 — No search engine can fix this
If a fact was never written into any summary, no amount of better searching can find it. Better ranking changes *which* summaries are read, never *what is inside them*. The problem is in the index, so the fix has to be in the index too.

2. Our solution in plain words

Think of video compression. A video doesn't store every frame in full: it stores **keyframes** plus small **residuals** that carry exactly what a keyframe couldn't. Nothing is lost, and the extra data stays small.

We do the same for GraphRAG. The community summaries are the keyframes. We add the residual: an exact record of everything the summaries left out.

1. **Audit the loss.** Compare the full relationship list with what the communities contain. This is a set comparison over files GraphRAG already writes — no AI calls, no guessing.
2. **Bridge records.** For the most important relationships lost at boundaries, add one short line to *both* communities' summaries: "SOURCE — relation — TARGET". Built from text GraphRAG already extracted, so no summary is rewritten.
3. **Graceful truncation.** Instead of cutting the list dead at the budget, write full descriptions until about 80% of the budget, then one-line compact facts for the rest. Nothing GraphRAG would have kept is displaced.
4. **Residual store.** Whatever still doesn't fit is stored next to the communities it touches. At answer time, the most relevant residual facts *replace* the weakest part of the context, so the number of

tokens stays exactly the same.

5. **Coverage certificate.** At build time we check that every relationship is in a summary, a bridge record, or the residual store. If one is unaccounted for, the build fails.

Same summaries. Same token budget. No extra AI calls. And the index can prove it didn't quietly forget anything.

3. What the simulation shows

A simulated index of a novel: 360 entities, 905 relationships, 48 questions, an 8,000-token summary budget, and the three most relevant summaries read per question. *These are simulated figures that show how the mechanism behaves — not experimental results.*

Across the hierarchy

		Relationships kept		Evidence found		Answers right
Level	**Communities**	GraphRAG	Ours	GraphRAG	Ours	GraphRAG → Ours
Leaf	22	72.6%	**78.8%**	49.2%	**75.7%**	41.7% → **56.3%**
Community	8	84.2%	**88.4%**	69.1%	**84.7%**	52.1% → **62.5%**
Higher community	3	46.3%	**65.2%**	80.2%	**87.0%**	60.4% → **66.7%**
Root	1	14.4%	**23.4%**	54.3%	**71.2%**	43.8% → **50.0%**

Read it like this. GraphRAG's own numbers swing wildly with the level — it keeps 84% of relationships one level up, but only 14% at the root, where one summary must cover everything. Our version is better at every level, and the gap widens exactly where compression bites hardest.

How much of the summary to spend on bridges

Bridge allowance (leaf)	**Relationships kept**	**Evidence found**	**Answers right**
0% (no bridges)	72.6%	49.2%	41.7%
6% of input	78.3%	75.7%	56.3%
12% of input	78.8%	75.7%	56.3%
24% of input	78.8%	75.7%	56.3%

The fix is cheap. Six percent of the summary buys the whole benefit. Spending more changes nothing, because there aren't many important boundary-crossing relationships to write down.

Which relationships to bridge

Bridge only if mentioned (leaf)	**Relationships kept**	**Evidence found**	**Answers right**
3+ times	85.0%	89.6%	66.7%
4+ times (our default)	78.8%	75.7%	56.3%
5+ times	75.8%	61.4%	43.8%
never (bridges off)	72.6%	49.2%	41.7%

This is the knob that matters. *Which* facts you bridge matters far more than how many tokens you allow. Loosening the rule from "mentioned 5 times" to "3 times" lifts correct answers from 43.8% to 66.7%, at the same cost.

Detail versus coverage

	Higher community		Root	
Full descriptions until	Kept	Evidence found	Kept	Evidence found
60% of budget	81.7%	89.2%	33.7%	89.1%
80% (our default)	65.2%	87.0%	23.4%	71.2%
100% (no compact facts)	48.0%	85.6%	14.5%	54.3%

Short beats absent. Switching to one-line facts earlier keeps far more of the graph. At the root, evidence found rises from 54% to 89% — the same budget, just spent on more facts in less detail.

Two things that don't help

Read more summaries	GraphRAG	Ours	**Buy a bigger budget**	GraphRAG	Ours
2 summaries	48.5%	74.5%	4,000 tokens	6.3%	11.6%
3 summaries	49.2%	75.7%	8,000 tokens	14.4%	23.4%
6 summaries	49.2%	76.2%	16,000 tokens	29.9%	48.8%

Left: evidence found at leaf level. Right: relationships kept at root level.

Reading three times as many summaries moves GraphRAG's evidence score by 0.7 of a point (48.5% $\rightarrow$ 49.2%). You cannot search your way out of this. Doubling the token budget does help — but you pay for it on every single query, while a residual costs nothing extra.

4. What we take from this

1. The loss is real at every level, and worst exactly where compression is heaviest.
2. Repair is cheap: about 6% of a summary's budget.
3. Choosing *which* facts to rescue matters more than the budget you give them.
4. A short fact beats a missing one: compact facts buy large coverage gains.
5. Better search doesn't help, which is the strongest sign this is an indexing problem.

5. Future work

1. **Prove the problem is worth solving.** Run the four stop criteria we committed to git before collecting any data: is the lost information already implied by summaries; do real questions need it; is indexing the cause of wrong answers; does restoring it actually change answers.
2. **Build it for real.** Implement the audit, bridge records, graceful truncation, residual store and coverage certificate as a pass over GraphRAG's own output files.
3. **Test it fairly.** Compare against GraphRAG, against a placebo that injects random lost facts of identical size, and against LightRAG — reporting results by question type.
4. **Tune the knobs on real data.** The bridge rule and the full-description cutoff are the two settings that mattered most here.
5. **Learn what to rescue.** Replace the hand-set rule with salience learned from past questions.
6. **Cover multi-topic entities.** Add pointers so an entity's other roles are visible in the communities it wasn't filed under.
7. **Replicate.** Repeat on a second corpus (medical) to show the result isn't specific to one dataset.

All figures here come from a simulation of the mechanism, not from an experiment on a real index. The stop criteria and evaluation plan above are how we intend to establish the result.